Практическая работа №1. Первое подключение C#-приложения к SQL Server

Цель работы

Научиться:

- создавать консольное приложение на C#;
- подключать пакет `Microsoft.Data.SqlClient`;
- создавать строку подключения;
- использовать `SqlConnection`;
- открывать соединение с SQL Server;
- выполнять простые SQL-команды через `SqlCommand`;
- получать одиночный результат с помощью `ExecuteScalar()`;
- проверять успешность подключения к базе данных.

Задание 1. Создание проекта

Создайте новое консольное приложение на C#.

Название проекта:

```
AdoNetPractice01
```

Если используется терминал:

```
dotnet new console -n AdoNetPractice01
```

Перейдите в каталог проекта:

```
cd AdoNetPractice01
```

Задание 2. Подключение Microsoft.Data.SqlClient

Установите пакет:

```
dotnet add package Microsoft.Data.SqlClient
```

После установки в программе подключите пространство имён:

```
using Microsoft.Data.SqlClient;
```

Проверьте, что проект собирается без ошибок.

Задание 3. Создание строки подключения

Создайте переменную со строкой подключения к вашей учебной базе данных SQL Server.

Например:

```
string connectionString =
    "Server=localhost;Database=CoffeeShop;Trusted_Connection=
True;TrustServerCertificate=True;";
```

Если SQL Server установлен под другим именем экземпляра, укажите его в параметре `Server`.

Например:

```
Server=localhost\SQLEXPRESS
```

или:

```
Server=.\SQLEXPRESS
```

Используйте параметры подключения, соответствующие вашей учебной установке SQL Server.

Задание 4. Создание SqlConnection

Создайте объект подключения:

```
using var connection = new SqlConnection(connectionString);
```

После этого откройте соединение:

```
connection.Open();
```

Добавьте сообщение:

```
Console.WriteLine("Соединение с базой данных установлено.");
```

Полный пример:

```
using Microsoft.Data.SqlClient;

string connectionString =
    "Server=localhost;Database=CoffeeShop;Trusted_Connection=
True;TrustServerCertificate=True;";

using var connection = new SqlConnection(connectionString);
```

```
connection.Open();

Console.WriteLine("Соединение с базой данных установлено.");
```

Запустите программу.

Если подключение прошло успешно, в консоли должно появиться:

```
Соединение с базой данных установлено.
```

Задание 5. Получение имени базы данных

Теперь проверим, действительно ли приложение работает с нужной базой данных.

Создайте SQL-команду:

```
string sql = "SELECT DB_NAME();";
```

Создайте объект `SqlCommand`:

```
using var command = new SqlCommand(sql, connection);
```

Выполните команду:

```
var databaseName = command.ExecuteScalar();
```

Выведите результат:

```
Console.WriteLine($"Текущая база данных: {databaseName}");
```

Полная программа:

```
using Microsoft.Data.SqlClient;

string connectionString =
    "Server=localhost;Database=CoffeeShop;Trusted_Connection=
True;TrustServerCertificate=True;";

using var connection = new SqlConnection(connectionString);

connection.Open();

Console.WriteLine("Соединение с базой данных установлено.");

string sql = "SELECT DB_NAME();";

using var command = new SqlCommand(sql, connection);

var databaseName = command.ExecuteScalar();

Console.WriteLine($"Текущая база данных: {databaseName}");
```

Проверьте результат.

Пример:

```
Соединение с базой данных установлено.
Текущая база данных: CoffeeShop
```

Задание 6. Получение информации о сервере

Измените SQL-запрос:

```
SELECT @@SERVERNAME;
```

Программа должна получить имя SQL Server и вывести его в консоль.

Например:

```
SQL Server: MY-PC\SQLEXPRESS
```

Для этого достаточно изменить значение:

```
string sql = "SELECT @@SERVERNAME;";
```

Остальной код можно оставить без изменений.

Задание 7. Выполнение нескольких запросов

Теперь последовательно получите три значения:

1. имя текущей базы данных;
2. имя SQL Server;
3. текущую дату и время сервера.

Для получения текущей даты SQL Server используется:

```
SELECT GETDATE();
```

Программа должна вывести примерно:

```
База данных: CoffeeShop
SQL Server: MY-PC\SQLEXPRESS
Дата и время сервера: 29.09.2026 21:45:32
```

Для каждого запроса можно создать отдельный `SqlCommand`.

Например:

```
using var command = new SqlCommand(
    "SELECT GETDATE();",
    connection);

var serverDate = command.ExecuteScalar();

Console.WriteLine($"Дата и время сервера: {serverDate}");
```

Задание 8. Получение количества товаров

Теперь перейдём от системной информации к данным нашей базы.

Если в базе есть таблица:

```
Products
```

выполните запрос:

```
SELECT COUNT(*)
FROM Products;
```

Получите результат через:

```
ExecuteScalar();
```

и выведите:

```
Количество товаров: 15
```

Количество будет зависеть от данных вашей базы.

Пример:

```
string sql = """
    SELECT COUNT(*)
    FROM Products;
    """;

using var command = new SqlCommand(sql, connection);

var productCount = command.ExecuteScalar();

Console.WriteLine($"Количество товаров: {productCount}");
```

Задание 9. Проверка работы с таблицей

Получите название и цену самого дорогого товара.

Используйте SQL:

```
SELECT TOP 1
    name,
    price
FROM Products
ORDER BY price DESC;
```

На этом этапе нам пока не нужно разбирать `SqlDataReader`.

Поэтому для проверки можно временно использовать запрос, который возвращает одно значение:

```
SELECT MAX(price)
FROM Products;
```

Получите максимальную цену через:

```
ExecuteScalar();
```

Выведите:

```
Максимальная цена товара: 450
```

Самостоятельная часть

Теперь выполните небольшое задание без пошаговой инструкции.

Ваша программа должна подключиться к учебной базе данных и вывести информацию о ней.

Необходимо получить:

1. название текущей базы данных;
2. имя SQL Server;
3. количество товаров;
4. минимальную цену товара;
5. максимальную цену товара.

Результат должен выглядеть примерно так:

```
=== Информация о базе данных ===

База данных: CoffeeShop
SQL Server: MY-PC\SQLEXPRESS

Количество товаров: 15
Минимальная цена: 120
Максимальная цена: 450
```

Для выполнения задания используйте:

```
SqlConnection
SqlCommand
ExecuteScalar()
```

Каждый SQL-запрос должен выполняться через `SqlCommand`.

Что необходимо продемонстрировать преподавателю

Готовая программа должна:

- успешно подключаться к SQL Server;
- использовать `Microsoft.Data.SqlClient`;
- использовать `SqlConnection`;
- использовать `SqlCommand`;
- использовать `ExecuteScalar()`;
- получать данные непосредственно из SQL Server;
- выводить полученные значения в консоль.

Выводы по работе

В ходе практической работы было создано C#-приложение, взаимодействующее с SQL Server через ADO.NET.

В работе были использованы:

```
Microsoft.Data.SqlClient
SqlConnection
SqlCommand
ExecuteScalar()
```

Было выполнено подключение к базе данных и несколько SQL-запросов.

В результате приложение научилось:

- устанавливать соединение с SQL Server;
- выполнять SQL-команды;
- получать одиночные значения из базы данных;
- использовать полученные данные в C#-программе.

Полученные знания являются основой для дальнейшей работы с ADO.NET.